

exp-fips203-mlkem-pke-encrypt-k2

FIPS 203 Alg.14 ML-KEM-512 PKE Encrypt 实现方案

ascendc 工程 (ML-KEM-512 W4a / E14)

2026-07-27

摘要

本文档描述 `examples/incubating/ml-kem/ml-kem-512/exp-fips203-mlkem-pke-encrypt-k2/` 在 Atlas A2 / Ascend910B4 上以 AscendC 实现 FIPS 203 Algorithm 14, 即 ML-KEM-512 ($k=2$) PKE Encrypt。

参数锁定: `docs/specs/fips203-mlkem512-parameter-card.md` 与 `ascendc-tests/ml-kem/ml-kem-512/pass-fix-f203-alg14-pke-encrypt-device-k2/STATUS.md`。生产 I/O: `input/` 为 `ek_pke.bin` (800B)、`m.bin` (32B)、`coins.bin` (32B) 与静态 LUT; `output/` 仅 `c.bin` (768B)。**计划 AI Core 数:** 2 launch; Launch 1 为 AIV_ONLY blockDim=2, Launch 2 为 MIX blockDim=1 (S-1)。**禁止:** 写入 `stable-512`; 从 `frozen/` 抄码; 用 k3/k4 的 `ek/c` 长度、`re[7/9]` 或零垫替代 k2 几何。

目录

1 工程边界	2
1.1 禁止项	2
2 数学语义 (Alg.14, $k=2$, $n=256$, $q=3329$)	2
2.1 输入输出	2
2.2 阶段公式	2
3 AscendC 实现规划	3
3.1 Launch 与 AI Core 规模	3
3.2 GM 几何	3
4 AscendC API 表	3
4.1 评估过但未采用	4
5 数学步骤覆盖率	5
6 验证计划	5
7 产出物	5

1 工程边界

来源	用途
ascendc-tests/ml-kem/ml-kem-512/pass-fix-f203-alg14-pke-encrypt-device-k2/	活跃 D14 k2 行为基线；可复制后自包含适配
examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-encrypt-k3/	活跃 k3 incubating 结构参考；仅 retarget 到 k2
library/shared/CANN / tikicpulib / CAModel	SHAKE/Keccak、统一整数压缩辅助等编译与运行环境

1.1 禁止项

- 禁止从 frozen/ 拿源码、customspec 或路线。
- 禁止将 $\hat{A}$ 、re、 u 、 v 等中间态作为默认 I/O 落盘。
- 禁止把噪声或 INTT 输入零垫到 6/8；re 锁定为 5 poly。
- 禁止新建或写入 stable-512。

2 数学语义 (Alg.14, $k = 2$, $n = 256$, $q = 3329$)

2.1 输入输出

张量 / 文件	契约
input/ek_pke.bin	800B；前 768B 为 $\hat{t}$ 的 ByteEncode_{12} ，末 32B 为 ρ
input/m.bin	32B 明文消息
input/coins.bin	32B Alg.14 随机性；用于 PRF 生成 $r\ e_1\ e_2$
input/lut_even_stacked.bin	NTT/INTT Stage2 LUT
lut_odd_stacked.bin	
output/c.bin	768B, $c = c_1\ c_2$, 其中 $c_1 = 640\text{B}$ ($d_u = 10$, 2 poly), $c_2 = 128\text{B}$ ($d_v = 4$, 1 poly)

2.2 阶段公式

1. 行 2: 从 ek_pke[0:768] 解码 $\hat{t} \in R_q^2$, 在 compute launch 内完成。

2. 行 3-7: 取 $\rho = \text{ek_pke}[768:800]$, 对 $(j, p) \in \{0, 1\}^2$ 生成 $\hat{A}[j, p]$, GM 锁定为 `[4, 256] int32`。
3. 行 8-15: 用 $\eta = 2$ 从 `coins` 采样 $r \|_{e_1 \| e_2}$ 共 5 个 poly; GM 锁定为 `[5, 256]`。
4. 行 16-18: $\hat{r} = \text{NTT}(r)$; $\hat{u}_j = \sum_p \hat{A}[j, p] \circ \hat{r}_p$; $\hat{tr} = \sum_p \hat{t}_p \circ \hat{r}_p$ 。
5. 行 19-21: 对 $\hat{u}_0, \hat{u}_1, \hat{tr}$ 与一空槽执行真 **INTT batch=4**; 得到 u_0, u_1, v 后分别加 e_1 与 $e_2 + \mu(m)$ 。
6. 行 22: $c_1 = \text{ByteEncode}_{10}(\text{Compress}_{10}(u))$, $c_2 = \text{ByteEncode}_4(\text{Compress}_4(v))$, 输出 $c = c_1 \| c_2$ 。

3 AscendC 实现规划

3.1 Launch 与 AI Core 规模

Launch	核类型 / blockDim	数据划分与理由
1 prep	AIV_ONLY, blockDim=2	$\hat{A}$ 共 4 poly 按 2+2; $r \ _{e_1 \ e_2}$ 共 5 poly 写真实 GM。继承 D14 k2 已验。
2 compute+tail	MIX, blockDim=1	S-1: $\text{NTT}(r)$ k=2 $\rightarrow$ 内积 $\rightarrow$ INTT batch4 (槽 u_0, u_1, v , 空) $\rightarrow$ Compress/ByteEncode $d = 10/4$ 。

3.2 GM 几何

对象	锁定几何
$\hat{A}$	<code>[4, 256] int32</code> ; 禁止 pad 到 9/16
<code>re</code>	<code>[5, 256] int32</code> ; <code>r[0..1] e1[0..1] e2[0]</code>
$\hat{r}$ NTT	<code>polyvec2</code> ; AIV 1+1
INTT batch4	<code>S0 [8, 256] int8</code> , <code>mat_c [32, 128] int32</code> ; 语义槽 u_0, u_1, v , 空
Tail pack	u 两 poly $d_u = 10 \rightarrow 640\text{B}$; v 一 poly $d_v = 4 \rightarrow 128\text{B}$

4 AscendC API 表

本实现不引入 E14 独有的新 AscendC API; 约束按 API 查阅索引执行。

API 名	分类 / 文档	Atlas A2	本用例 dtype	备注
-------	---------	----------	--------------	----

GetBlockIdx / GetSubBlockIdx	系统变量 / 资源	√	标量索引	区分 AIV 分片与 MIX 子核
GlobalTensor / LocalTensor	基础结构	√	uint8/int8/int32	63B/UB 张量契约须与本 spec 几何一致
DataCopy	Memory 数据搬运	√	uint8/int8/int32	连续块搬运; 32B 对齐约束
Duplicate	Memory 矢量计算	√	uint8/int32	UB 清零、pad 与 tail buffer 初始化
PipeBarrier	Pipe 同步	√	—	MTE/Vector/Cube 阶段交界显式同步
CrossCoreSetFlag / CrossCoreWaitFlag	MIX 同步	√	—	AIV/AIC 通过 GM 交接; CPU 过不能删
Mmad / Fixpipe	矩阵计算	√	int8*int8->int32	88B 2 MMAD; 硬件 M 对齐垫不表示假 poly
ShiftRight	矢量算术	√	int32	limb 拆分、Barrett 与 ByteEncode 辅助右移
Muls / Add / Sub	矢量算术	√	int32/int16	limb、模加减、压缩/编码辅助
Cast	类型转换	√ (受限)	int32/int16/int8	128B 128 Cast 链; 不得假设 int32->int8 单跳
Gather	矢量搬运/选择	√	int32	仅允许非 NTT S1-S3 段; NTT 三段内禁用
GetValue / SetValue	LocalTensor 标量访问	√	uint8/int32	仅用于 pack 尾部等标量缺口

4.1 评估过但未采用

API / 路线	不采用原因
零垫 re 到 7/9	T-B2 / 禁零垫; D14 k2 已锁 5 poly
Gather 用于 NTT S1-S3	NTT 定稿禁止

高阶 Matmul<>

继承 D14 手写 MMAD + 平面 mat_c

5 数学步骤覆盖率

数学步骤	实现方式 / API	覆盖	缺口说明
解码 $\hat{t}$ 、取 ρ	compute 内 ByteDecode12 + 指针 切片	部分向量	unpack 标量缺口 沿用 D14
采样 $\hat{A}[4]$	AIV_ONLY + XOF/rej	100% 设备	2+2; 无零垫
采样 $r\ e_1\ e_2[5]$	AIV_ONLY + CBD $\eta = 2$	100% 设备	真实 5 poly
NTT / 内积 / INTT batch4	MIX + Mmad	100% 设备	NTT 内无 Gather; 空槽不 写假 poly 语义
Compress/ByteEncode $d = 10/4$	统一整数压缩 + pack	部分向量	pack 尾部标量缺 口沿用 D14

6 验证计划

```
cd examples/incubating/ml-kem/ml-kem-512/exp-fips203-mlkem-pke-encrypt-k2
bash run.sh -r cpu -v Ascend910B4
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

期望: c.bin 为 768B, 与 golden 对拍 PASS; 用例根无 stray dump。

7 产出物

- exp-fips203-mlkem-pke-encrypt-k2-实现方案-customspec.tex
- exp-fips203-mlkem-pke-encrypt-k2-实现方案-customspec.pdf
- 后续实现文件须与本 customspec 同目录自包含, 且自研 Python/C/AscendC 写详细中文注释。